

# Continuous–Joint Wraparound in the Compliant Effort Controller

A mathematical analysis of the “random spinning” failure

compliant\_controllers — Kinova Gen3 (7 DoF)

June 22, 2026

## Abstract

This note derives, from first principles, why a joint can occasionally “run away” (spin a full turn) when the arm is driven by the compliant *effort* controller. The mechanism is a mismatch between the angle convention used for the *measured* state and the convention used for the *desired* (commanded) state: the measured state is unwrapped into a continuous frame with a per-joint integer offset, while the desired state is consumed in the trajectory controller’s own (raw) frame. The resulting constant  $2\pi$  position error is multiplied by the joint stiffness and applied as an *unclamped* torque, which drives the joint around. We give the algebra, identify the exact source terms in the code, separate what is verified from what is assumed, and state the fix.

## Contents

### 1 Scope and honesty notes

The controller specialises `hardware_interface::EffortJointInterface` only; there is no position/velocity variant. Hence the analysis below applies exactly when the arm is in *effort control mode*. In any position mode the control law of Section ?? does not run and the robot’s onboard controller resolves wraparound, which is consistent with the observation that the symptom appears only in effort mode.

Two facts are **verified directly from the source tree**:

- the measured state receives a one-shot  $+2\pi$  shift and is then unwrapped into a continuous frame (Section ??);
- the desired state is consumed *raw* and the proportional position error is *unclamped* (Section ??).

One ingredient is **assumed and flagged for verification** (Section ??): the precise angle convention of (a) the upstream `joint_trajectory_controller` for `continuous` joints and (b) the kortex hardware driver’s reported range. These determine *how often* the failure offset is non-zero, not *whether* the mechanism exists.

## 2 Notation

For joint  $i$  (the `continuous` joints on the Gen3 are  $i \in \{1, 3, 5, 7\}$ ):

$$\begin{aligned}\phi_i^m(t) &\in \mathbb{R} && \text{true (unbounded) measured physical angle,} \\ \phi_i^d(t) &\in \mathbb{R} && \text{true (unbounded) desired physical angle,} \\ q_i^{hw}(t) &&& \text{hardware reading} \equiv \phi_i^m \pmod{2\pi}, \text{ wrapped,} \\ \Delta t &&& \text{control period,} \quad \Theta = \frac{3\pi}{2} \text{ wrap threshold.}\end{aligned}$$

The normalisation used by the code (`src/extended_joint_positions.cpp:96`) is

$$\mathcal{N}(x) = ((x + \pi) \bmod 2\pi) - \pi \in [-\pi, \pi). \quad (1)$$

The adapter shift (`hardware_interface_adapter_impl.h:140`), applied once per cycle to every joint, is

$$\tilde{q}_i = q_i^{hw} + 2\pi \lfloor q_i^{hw} < 0 \rfloor, \quad \tilde{q}_i \equiv \phi_i^m \pmod{2\pi}. \quad (2)$$

## 3 The extended (measured) position

With stored raw value  $c_i$  (`current_`) and accumulator  $\theta_i$  (`diff_`), one `update(t)` performs

$$\theta_i \leftarrow \begin{cases} \theta_i + \mathcal{N}(t) - \mathcal{N}(c_i), & |t - c_i| \geq \Theta \quad (\text{“wrap”}), \\ 2\pi \lfloor \theta_i / 2\pi \rfloor + t, & |t - c_i| < \Theta \quad (\text{“no wrap”}), \end{cases} \quad c_i \leftarrow t. \quad (3)$$

The integer count in the source — `static_cast<int>(θ/2π)` with a  $-1$  correction for negatives — equals  $\lfloor \theta_i / 2\pi \rfloor$ .

**Invariant.** As long as the per-cycle step stays small,  $\theta_i(t)$  is continuous and

$$\theta_i(t) \equiv \tilde{q}_i(t) \equiv \phi_i^m(t) \pmod{2\pi}. \quad (4)$$

Two continuous functions congruent mod  $2\pi$  differ by a fixed integer multiple of  $2\pi$ , frozen at initialisation:

$$\boxed{\theta_i(t) = \phi_i^m(t) + 2\pi k_i^0, \quad k_i^0 = \frac{\tilde{q}_i(0) - \phi_i^m(0)}{2\pi} \in \mathbb{Z}.} \quad (5)$$

From (??),  $k_i^0 = \lfloor q_i^{hw}(0) < 0 \rfloor$  at start (plus any later  $\pm 1$  injected by a misfire, Section ??).

## 4 The desired position and the control law

The desired trajectory state is copied *raw* into the controller (`hardware_interface_adapter_impl.h:149`); it is *not* passed through (??) or (??). Write its frame as

$$q_i^d(t) = \phi_i^d(t) + 2\pi m_i(t), \quad (6)$$

where  $m_i(t) \in \mathbb{Z}$  is the (integer) branch chosen by the trajectory controller’s own unwrapping convention (Section ??).

The effort law (`src/joint_space/controller.cpp:196`) is

$$\boldsymbol{\tau} = -K(\boldsymbol{\theta}_{\text{prev}}^{\text{nom}} - \boldsymbol{q}^d - K_s^{-1}\boldsymbol{g}) - D(\dot{\boldsymbol{\theta}}_{\text{prev}}^{\text{nom}} - \dot{\boldsymbol{q}}^d), \quad (7)$$

with  $K$  the joint stiffness,  $D$  the damping,  $K_s^{-1}$  the inverse joint stiffness and  $\boldsymbol{g}$  gravity. The proportional position error is  $\boldsymbol{e} = \boldsymbol{\theta}_{\text{prev}}^{\text{nom}} - \boldsymbol{q}^d$ . Only the *integral* state  $\boldsymbol{q}_{\text{error,sum}}$  is clamped (`controller.cpp:273`);  $\boldsymbol{e}$  itself is **not** clamped.

## 5 Error decomposition

In tracking  $\theta_i^{\text{nom}} \approx \theta_i$ , so combining (??) and (??),

$$\begin{aligned} e_i &= \theta_i - q_i^d = (\phi_i^m + 2\pi k_i^0) - (\phi_i^d + 2\pi m_i) \\ &= \underbrace{(\phi_i^m - \phi_i^d)}_{\text{true tracking error}} + 2\pi(k_i^0 - m_i(t)). \end{aligned} \quad (8)$$

The first term is what we want. The second is a spurious integer number of full turns, present whenever the measured frame ( $k_i^0$ , set by the  $+2\pi$  shift) and the desired frame ( $m_i$ , set by the trajectory controller) disagree.

## 6 Why it spins, and only in effort mode

The proportional torque on joint  $i$  is

$$\tau_i \supset -K_{ii}e_i = -K_{ii}(\phi_i^m - \phi_i^d) - \underbrace{2\pi K_{ii}(k_i^0 - m_i(t))}_{\tau_i^{\text{spurious}}}. \quad (9)$$

Each unit of  $k_i^0 - m_i$  contributes a *constant* torque of magnitude  $2\pi K_{ii}$  in a fixed direction. Because  $e$  is unclamped, this reaches the actuator in full. As the joint accelerates,  $\phi_i^m$  runs away but  $\theta_i$  tracks it continuously via (??), so  $e_i$  does *not* shrink — the “unwinding” never completes and the joint sustains rotation. In a position mode this law never executes, hence the effort-mode exclusivity.

## 7 The threshold misfire (the intermittency)

The wrap branch of (??) triggers on  $|t - c_i| \geq \Theta = \frac{3\pi}{2}$ . Let the effective per-cycle increment be  $\Delta_i = |\dot{\phi}_i^m| \Delta t_{\text{eff}}$ .

- **Correct** unwrapping requires  $\Delta_i < \frac{\pi}{2}$  (a genuine  $2\pi$  wrap then yields  $|t - c_i| = 2\pi - \Delta_i \geq \frac{3\pi}{2}$ ).
- A **false wrap** occurs when  $\Delta_i \geq \frac{3\pi}{2}$  with no boundary crossing: a large in-band step is misread as a wrap, and

$$\theta_i \mapsto \mathcal{N}(t) - \mathcal{N}(c_i) = (t - c_i) \mp 2\pi, \quad (10)$$

injecting  $\mp 2\pi$ , i.e. flipping  $k_i^0 \rightarrow k_i^0 \mp 1$  in (??)–(??).

With the Gen3 limit  $|\dot{\phi}_i^m| \leq 1.3963 \text{ rad/s}$ , a false wrap needs  $\Delta t_{\text{eff}} \geq \frac{3\pi/2}{1.3963} \approx 3.4 \text{ s}$  in a single cycle — i.e. a dropped or severely late real-time cycle. This is intermittent by nature and accounts for “every now and then.”

## 8 The unverified link: trajectory and driver conventions

Equation (??) shows the offset is  $k_i^0 - m_i(t)$ . The value of  $m_i(t)$  depends on the upstream `joint_trajectory_controller`: for continuous joints (`angle_wraparound = true`) it shifts each waypoint by the multiple of  $2\pi$  that makes the path continuous with the *current measured position*, then samples a continuous desired. Thus  $m_i$  is a (slowly varying / piecewise constant) integer

chosen about the *raw*  $q_i^{hw}$  origin — *not* about the  $+2\pi$ -shifted, extended  $\theta_i$  origin used by the compliant controller. The two origins differ by exactly  $k_i^0$  from (??).

How often  $k_i^0 - m_i \neq 0$  depends on the kortex driver's reported range:

- driver reports  $[0, 2\pi)$ : the shift (??) never fires,  $k_i^0 = 0$  initially, and the residual failures come from the misfire/drift path of Section ??;
- driver reports  $[-\pi, \pi)$ : the shift fires for any joint sitting negative, giving a near-deterministic  $2\pi$  offset right after start.

**To verify:** the installed JTC source (waypoint wraparound origin) and the kortex hardware interface's reported joint range. These set the *frequency*, not the *existence*, of the failure.

## 9 Fix

Two independent changes; either alone removes most of the risk, both together close it.

**(1) Frame-consistent desired.** Before differencing, replace  $q_i^d$  with the representative congruent to  $q_i^d$  and nearest to  $\theta_i^{\text{nom}}$ :

$$\hat{q}_i^d = q_i^d + 2\pi \text{round}\left(\frac{\theta_i^{\text{nom}} - q_i^d}{2\pi}\right), \quad e_i = \theta_i^{\text{nom}} - \hat{q}_i^d \in [-\pi, \pi]. \quad (11)$$

This cancels the  $2\pi(k_i^0 - m_i)$  term in (??) identically, leaving  $e_i = \mathcal{N}(\phi_i^m - \phi_i^d)$  — the true error taken the short way.

**(2) Clamp the proportional error.** As an independent safety net, bound  $e$  the way the integral term is already bounded:

$$e_i \leftarrow \text{clip}(e_i, -e_i^{\max}, e_i^{\max}), \quad (12)$$

so even a residual misfire can never command more than  $K_{ii}e_i^{\max}$ .

**(3) Robuster wrap detection (optional).** Replace the fixed threshold  $\Theta = \frac{3\pi}{2}$  by a velocity/period aware test, e.g. flag a wrap only when  $|t - c_i| > 2\pi - \kappa$  with  $\kappa$  a margin larger than the worst plausible  $\Delta_i$ , removing the false-wrap path of Section ??.

The same pattern (measured unwrapped, desired raw, unclamped  $e$ ) exists in the `joint_task_space` and `task_space` controllers and should receive the same treatment.